

NumPy, Pandas and Scikit-learn Basics

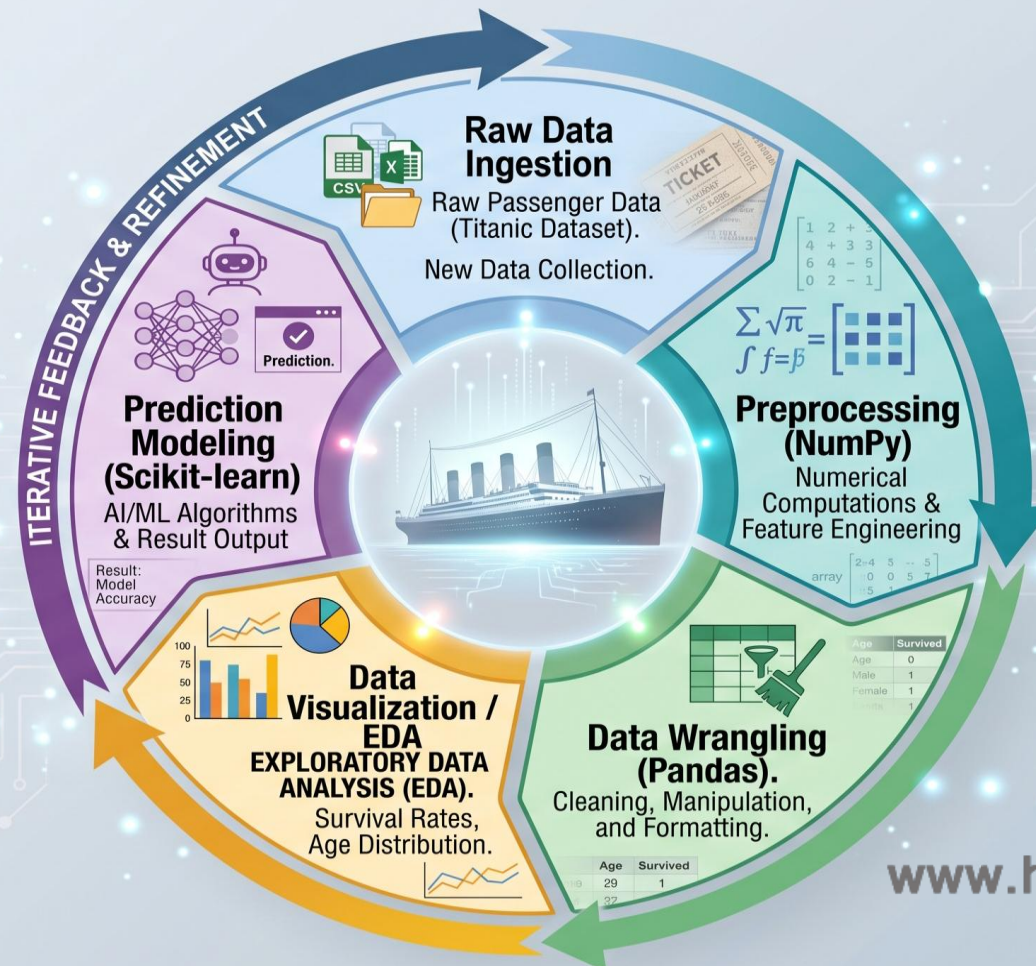
From Numerical Arrays to Machine Learning

Presented by

RA. Muhammad Huzifa

Supervisor

Dr. Muhammad Sajjad



www.huaweicloud.com

Learning Outcomes

CONCEPT EXPLANATION

By the end of this week, students should be able to:

1. Create and manipulate NumPy arrays.
2. Apply indexing, slicing, and reshaping.
3. Use broadcasting and matrix multiplication.
4. Load, explore, and clean data with Pandas.
5. Calculate basic statistics with NumPy.
6. Visualize data using Matplotlib or Seaborn.
7. Explain the basic Scikit-learn workflow.

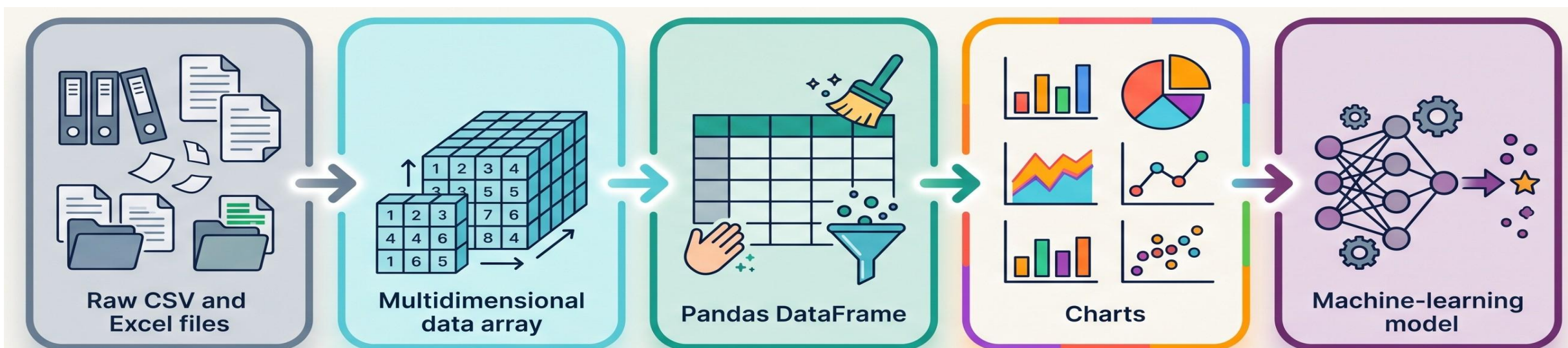
Complete Data Workflow

Simple concept explanation

A data project moves through several stages, starting with raw data and ending with analysis, visualization, or machine-learning predictions.

Complete Data Workflow

- 1.Raw Data:** Collect data from CSV or Excel files.
- 2.NumPy:** Perform numerical calculations.
- 3.Pandas:** Organize, clean, and transform data.
- 4.Visualization:** Discover patterns using charts.
- 5.Scikit-learn:** Train and evaluate a model.



What Is NumPy?

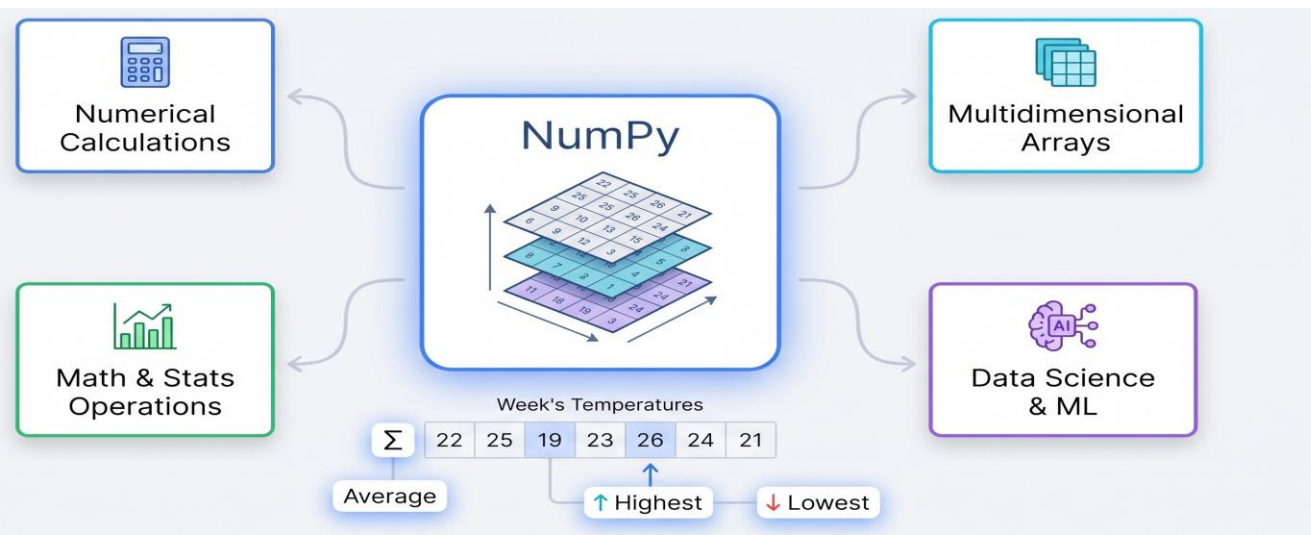
NumPy stands for **Numerical Python**. It stores numbers in structured arrays and makes numerical calculations easier and faster.

What Is NumPy?

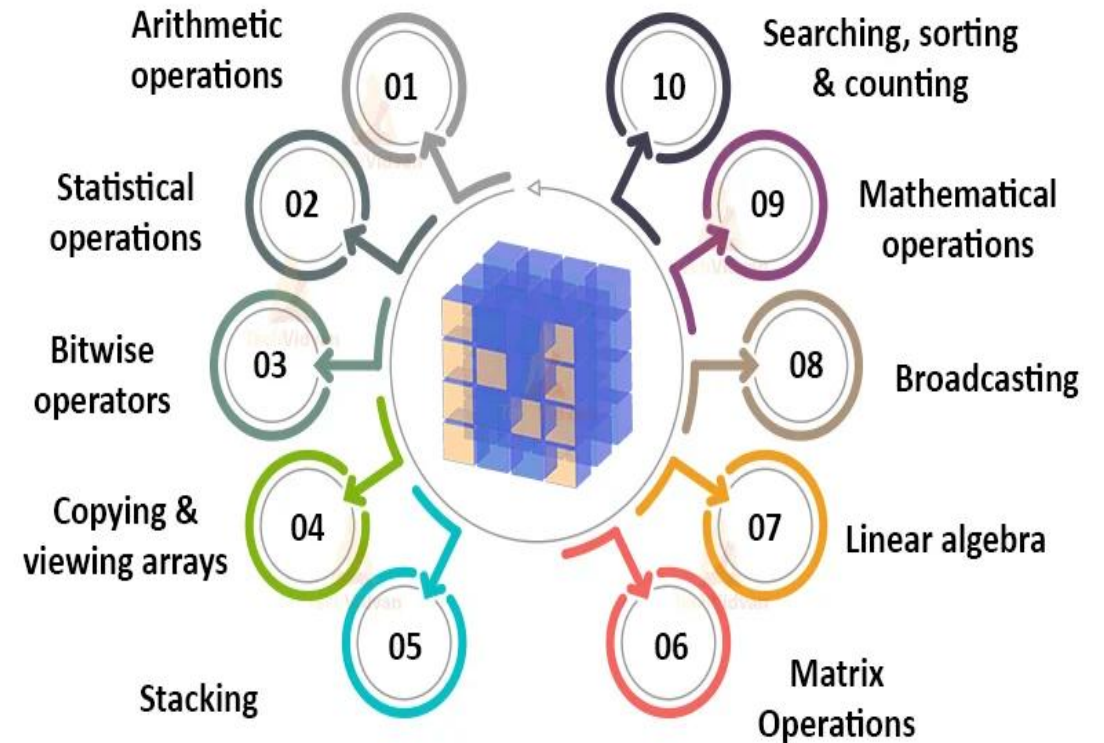
- A Python library for numerical calculations
- Stores data in multidimensional arrays
- Performs mathematical and statistical operations
- Commonly used in data science and machine learning

Example:

A week's temperatures can be stored in one NumPy array. We can then find the average, highest, and lowest temperature.



Uses of NumPy



Python Lists vs NumPy Arrays

Difference Between List and NumPy

List	NumPy
Basic data structure	Specialized data structure for numerical data
Can store elements of any data type	Typically stores elements of the same data type
Created using square brackets []	Created using NumPy functions or arrays
Example: <code>my_list = [1, 1, 3]</code>	Example: <code>import numpy as np a1 = np.array([1, 2])</code>
Slower for numerical operations	Designed for fast numerical operations
More memory consumption	Less memory consumption
No broadcasting for element-wise operations	Supports broadcasting for element-wise operations

Array Dimensions and Properties

An array's dimensions describe how its values are arranged. Its properties tell us about its structure and data type.

Array Dimensions

- **1D:** A single line of values
- **2D:** Values arranged in rows and columns
- **3D:** A stack of two-dimensional arrays

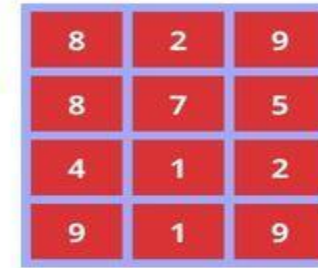
Main Properties

- **ndim:** Number of dimensions
- **shape:** Size of each dimension
- **size:** Total number of values
- **dtype:** Type of stored values

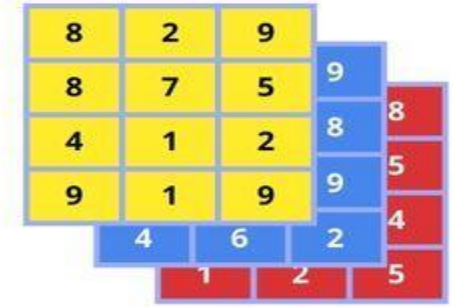
The marks of four students in three subjects form a 2D array. Its shape is 4×3 , and its size is 12.



1



2



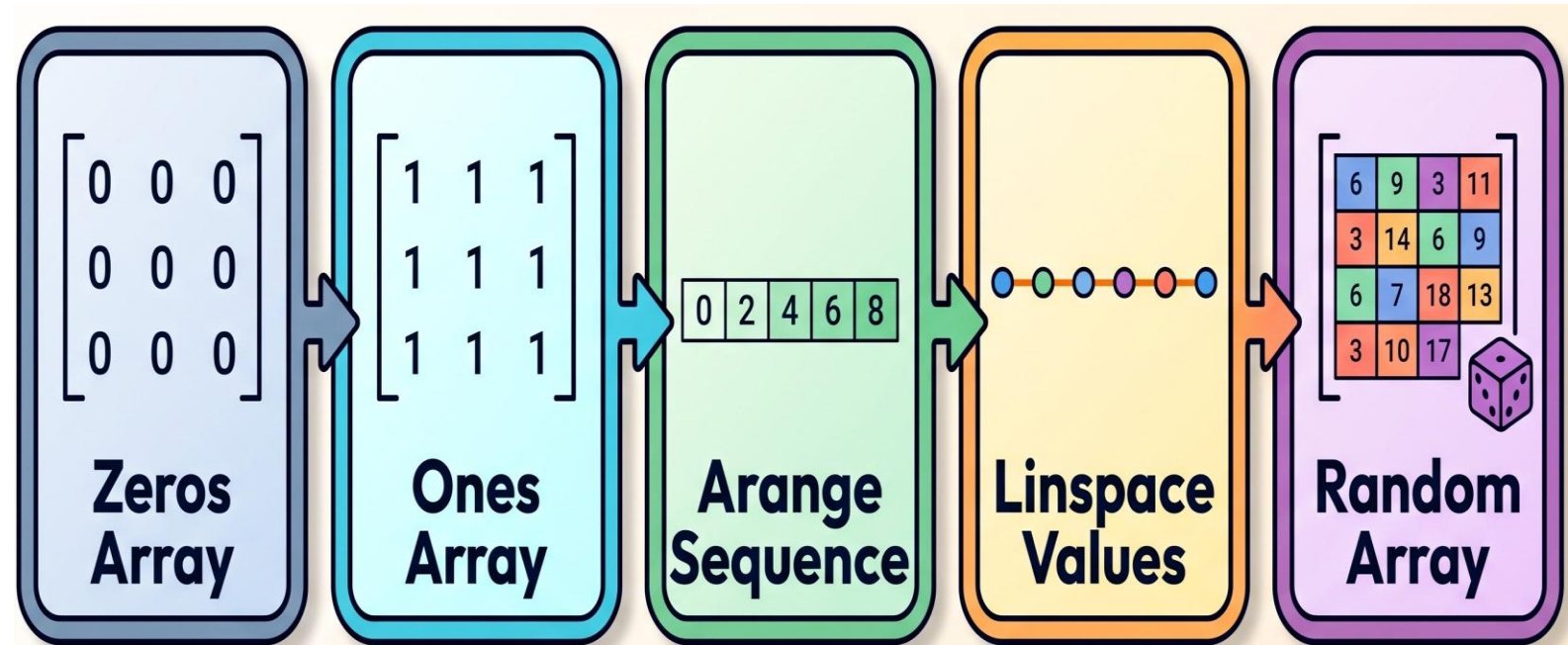
3

Creating Special Array

NumPy can create ready-made arrays without entering every value manually. These arrays are useful for initialization, sequences, and sample data.

Common Array-Creation Methods

- **Zeros:** Creates an array filled with 0
- **Ones:** Creates an array filled with 1
- **Arange:** Creates values using a fixed step
- **Linspace:** Creates evenly spaced values
- **Random:** Creates random values for testing



A teacher can create an empty marks table filled with zeros and update the values after checking each student's paper.

Indexing and Slicing

Indexing selects one value from an array. Slicing selects a group of values, rows, or columns.

Indexing

- Selects one value using its position
- Positions begin from 0
- A 2D array uses row and column positions

Slicing

- Selects a range of values
- The start position is included
- The stop position is excluded

For the array **[10, 20, 30, 40, 50]**:

- Position 2 gives **30**
- A slice from position 1 to 4 gives **[20, 30, 40]**



Reshaping , Flattening and Transposing

Reshaping

- Changes the number of rows and columns
- Total values must remain the same

Flattening

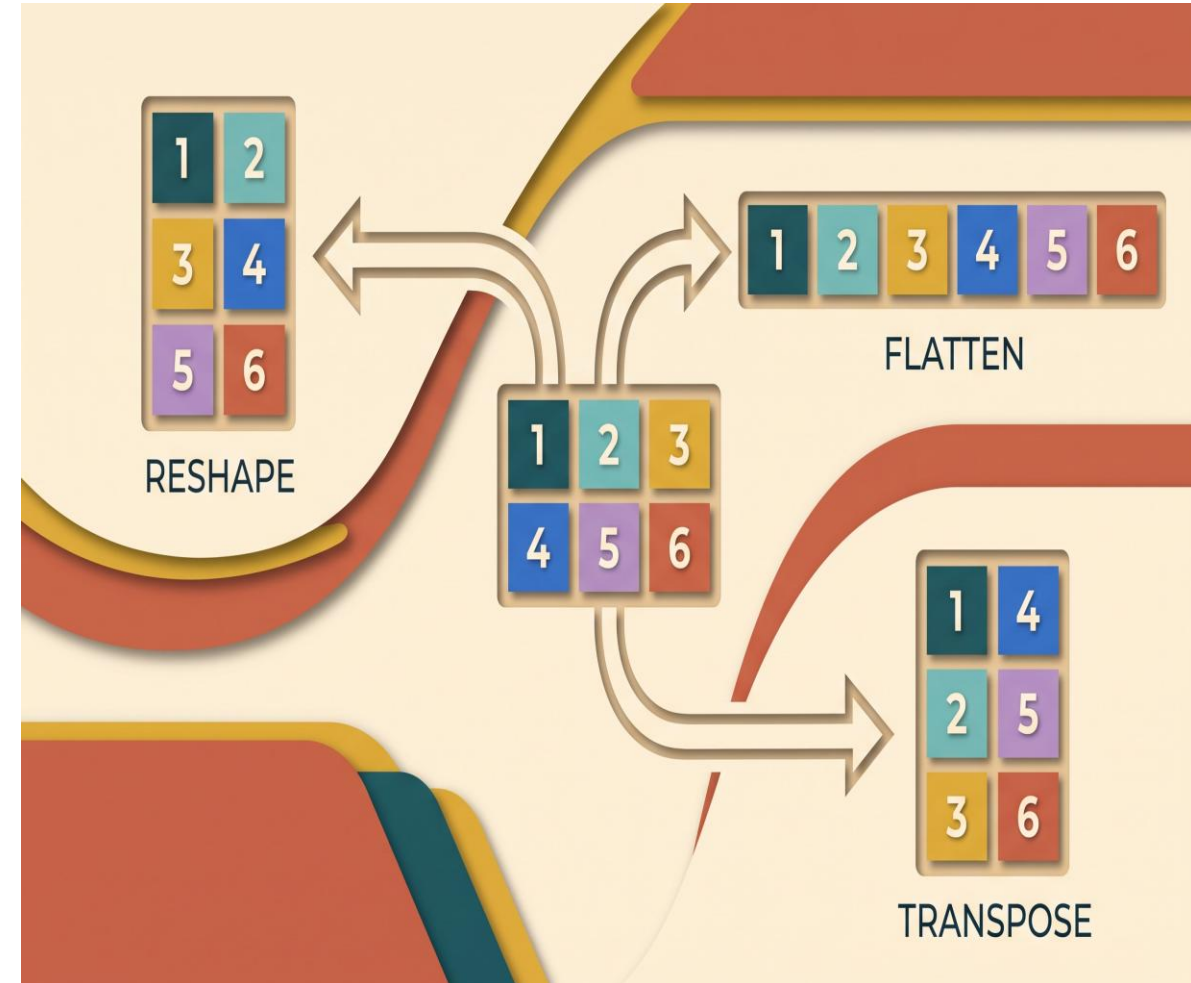
- Converts a multidimensional array into one row

Transposing

- Changes rows into columns and columns into rows

A 2×3 array can become:

- 3×2 through reshaping
- 1×6 through flattening
- 3×2 with exchanged rows and columns through transposing



Broadcasting in NumPy

Broadcasting allows NumPy to perform calculations using arrays of different but compatible shapes.

Broadcasting

- Matches a smaller value or array with a larger array
- Applies one operation across many values
- Requires compatible dimensions
- Reduces repeated data and calculations



Matrix Multiplications

Matrix multiplication combines the rows of one matrix with the columns of another to create a new matrix.

How It Works

- Select one row from the first matrix
- Match it with one column from the second matrix
- Multiply corresponding values and add the results
- Repeat the process for every output position

Compatibility Rule

- The inner dimensions must be equal
- A 3×2 matrix can multiply a 2×1 matrix
- The result will have shape 3×1

}

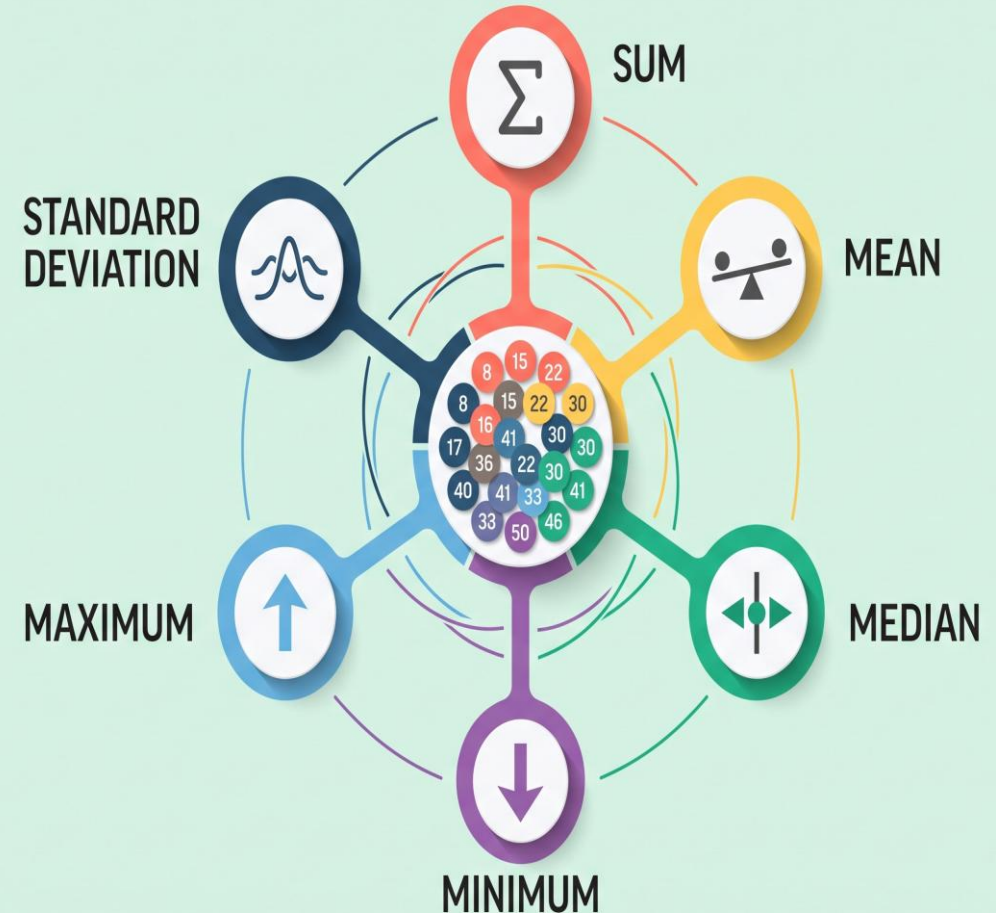
Numerical and Statistical Operations

NumPy can quickly calculate totals, averages and other useful information from numerical data.

Common Operations

- **Sum:** Adds all values
- **Mean:** Calculates the average
- **Median:** Finds the middle value
- **Minimum:** Finds the smallest value
- **Maximum:** Finds the largest value
- **Standard Deviation:** Measures how spread out the values are

Operations can be applied to the complete array, individual rows or columns.



What is Pandas

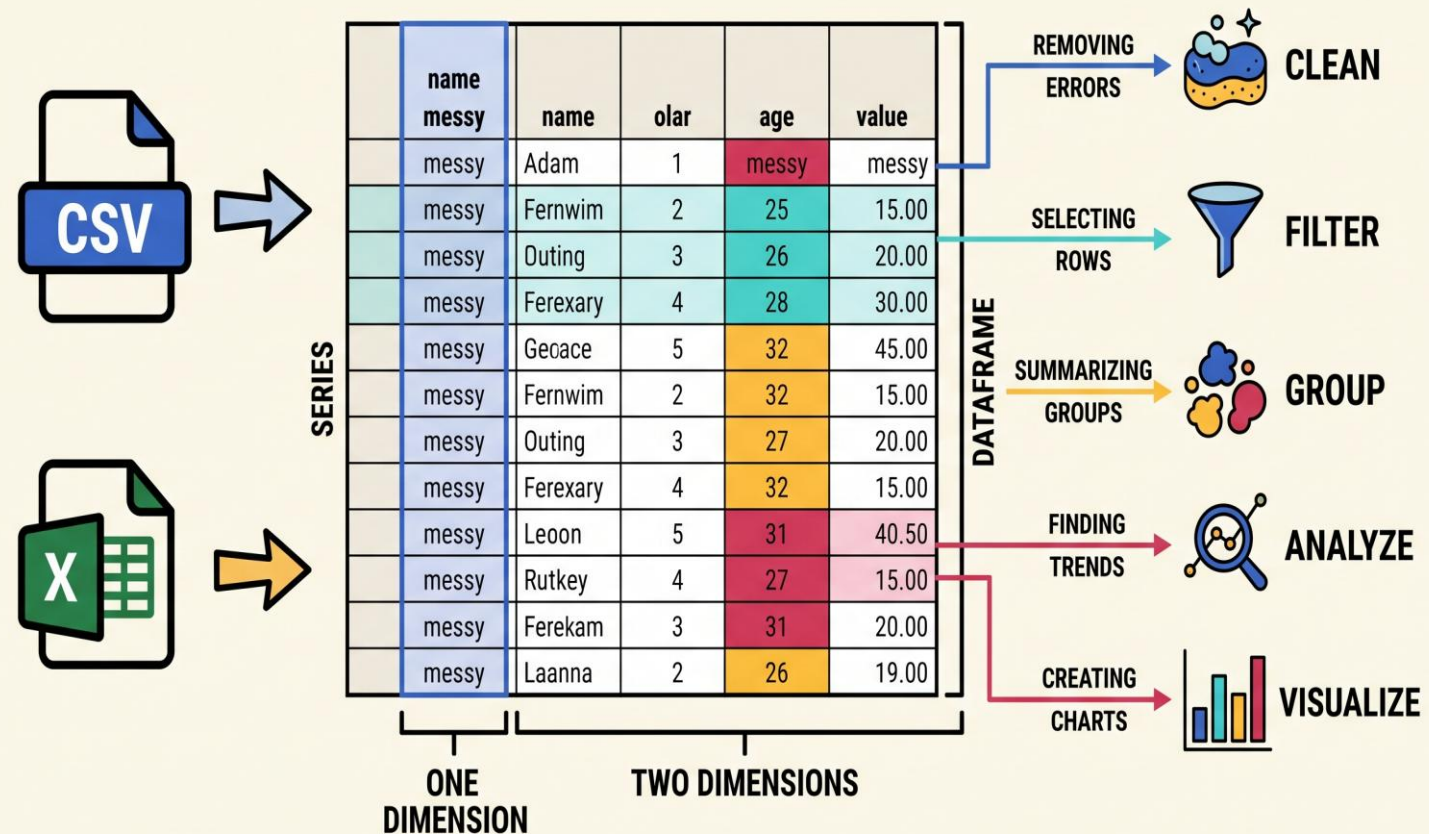
Pandas is a Python library used to organize, clean and analyze data arranged in rows and columns.

Pandas Can

- Load CSV and Excel files
- Organize data in tables
- Select and filter records
- Handle missing values
- Group and summarize data
- Prepare data for visualization and machine learning

Main Data Structures

- **Series:** One-dimensional labelled data
- **DataFrame:** Two-dimensional table



Pandas



	A	B	C
0			
1			
2			

Pandas Series and DataFrame

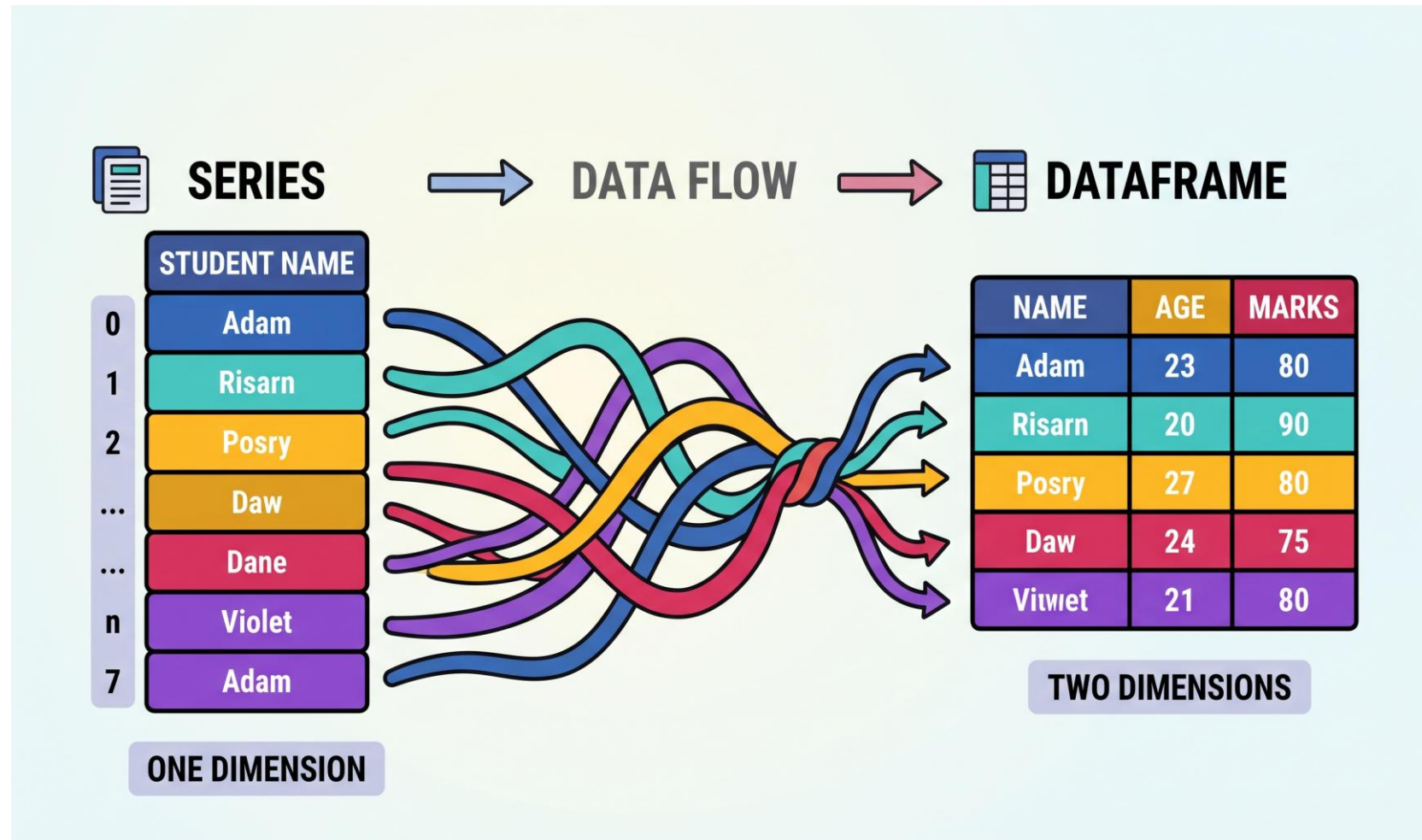
A Series represents one labelled column, while a DataFrame combines several columns into a complete table.

Series

- One-dimensional data
- Contains values and index labels
- Represents a single row or column

DataFrame

- Two-dimensional data
- Contains rows and columns
- Can store different types of data
- Each column is a Series



Loading CSV and Excel Files

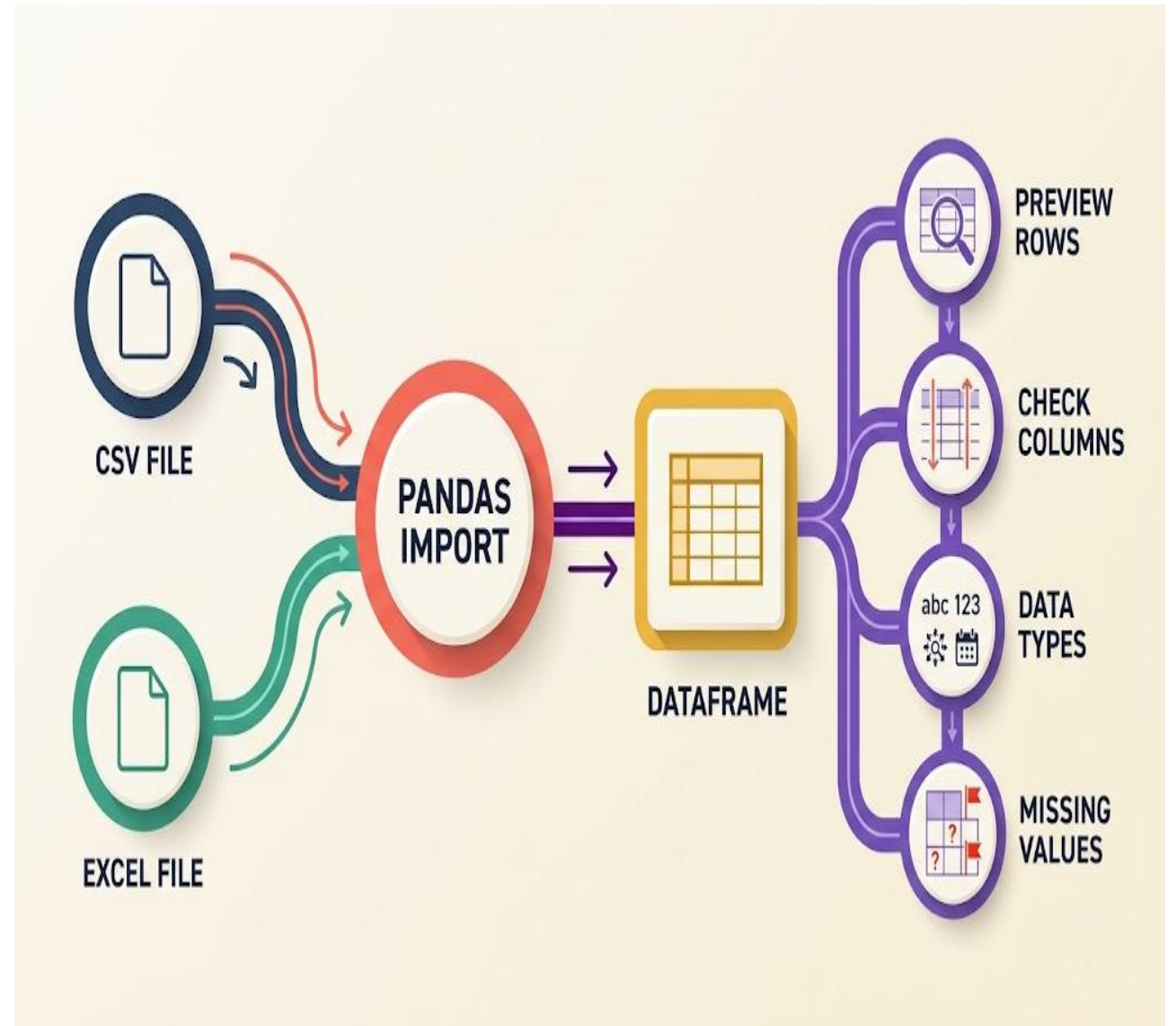
Pandas reads data from files and converts it into a DataFrame for exploration and analysis.

Common File Types

- **CSV:** A text file containing tabular data
- **Excel:** A spreadsheet containing one or more sheets

After Loading

- Data becomes a DataFrame
- Preview the first few rows
- Check column names and data types
- Identify missing values



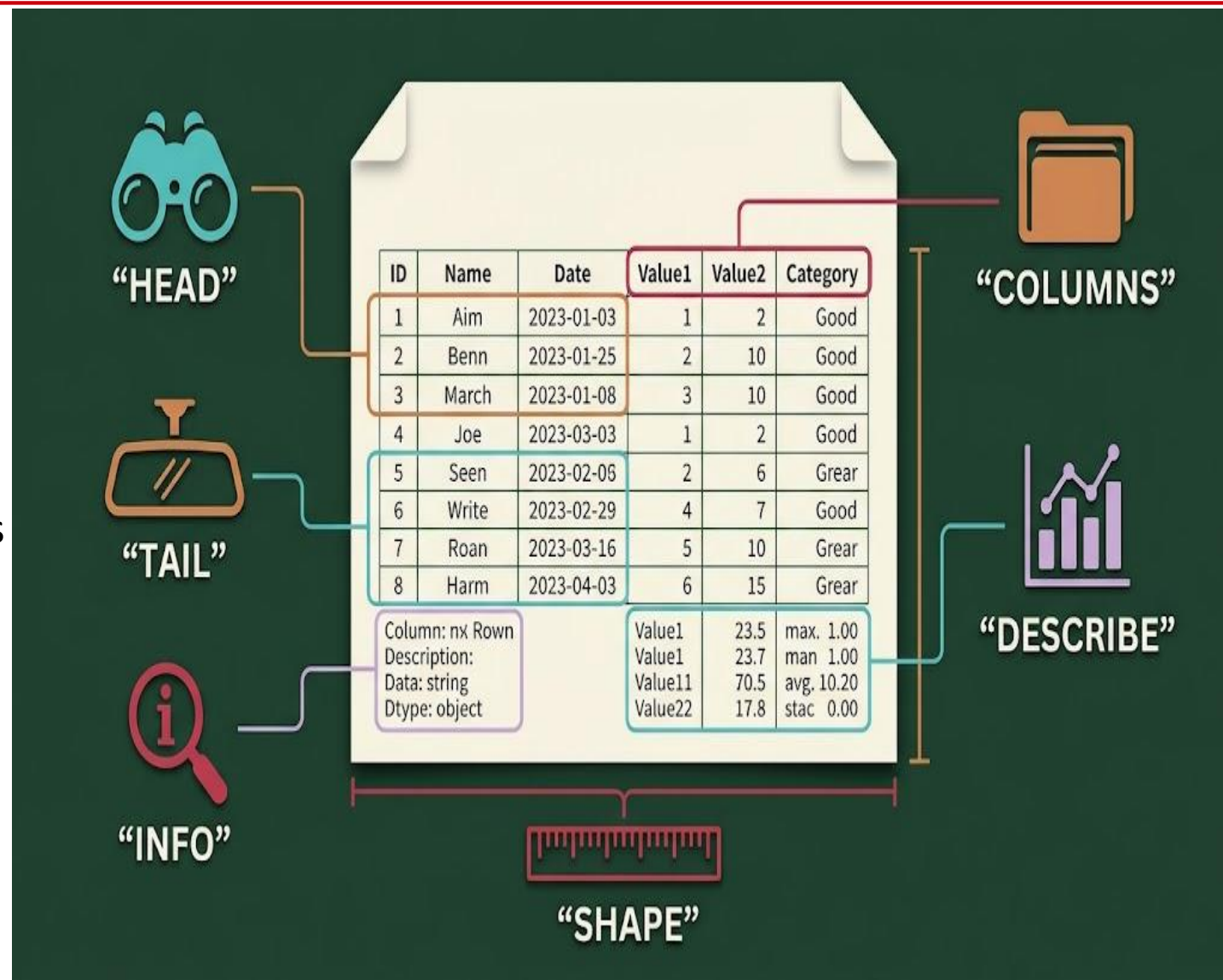
Exploring a DataFrame

Exploring a DataFrame helps us understand its structure, values and possible data-quality problems before analysis.

Useful Exploration Tools

- **head():** Displays the first rows
- **tail():** Displays the last rows
- **shape:** Shows the number of rows and columns
- **columns:** Lists all column names
- **info():** Shows data types and non-missing values
- **describe():** Provides a statistical summary

Before analyzing student records, check the first rows, dataset size, column names, data types and any missing values.



Selecting and Filtering Data

Selecting Data

- Choose one or more columns
- Use **loc** to select by labels
- Use **iloc** to select by position

Filtering Data

- Keep rows that match a condition
- Combine conditions for precise results
- Remove unrelated records from the view

From a student DataFrame, select the Name and Marks columns and keep only students who scored 70 or above

 SELECT COLUMNS

NAME	AGE	CITY	MARKS
Azam	23	Canada	70
John	24	Mina	70
Meter	26	Nanda	80
Joiry	28	Colli	60
Jares	30	Happa	70
Avan	29	Palis	65
John	20	Lincolan	70

 APPLY CONDITION

NAME	AGE	CITY	MARKS
Azam	25	Canada	70
John	24	Mina	70
Meter	26	Nanda	50
Joiry	28	Colli	70
Jares	30	Happa	70
Avan	29	Palis	65
John	30	Lincolan	40

FILTERED DATA

NAME	MARKS
Aram	70
John	70
Meter	60
Jares	70

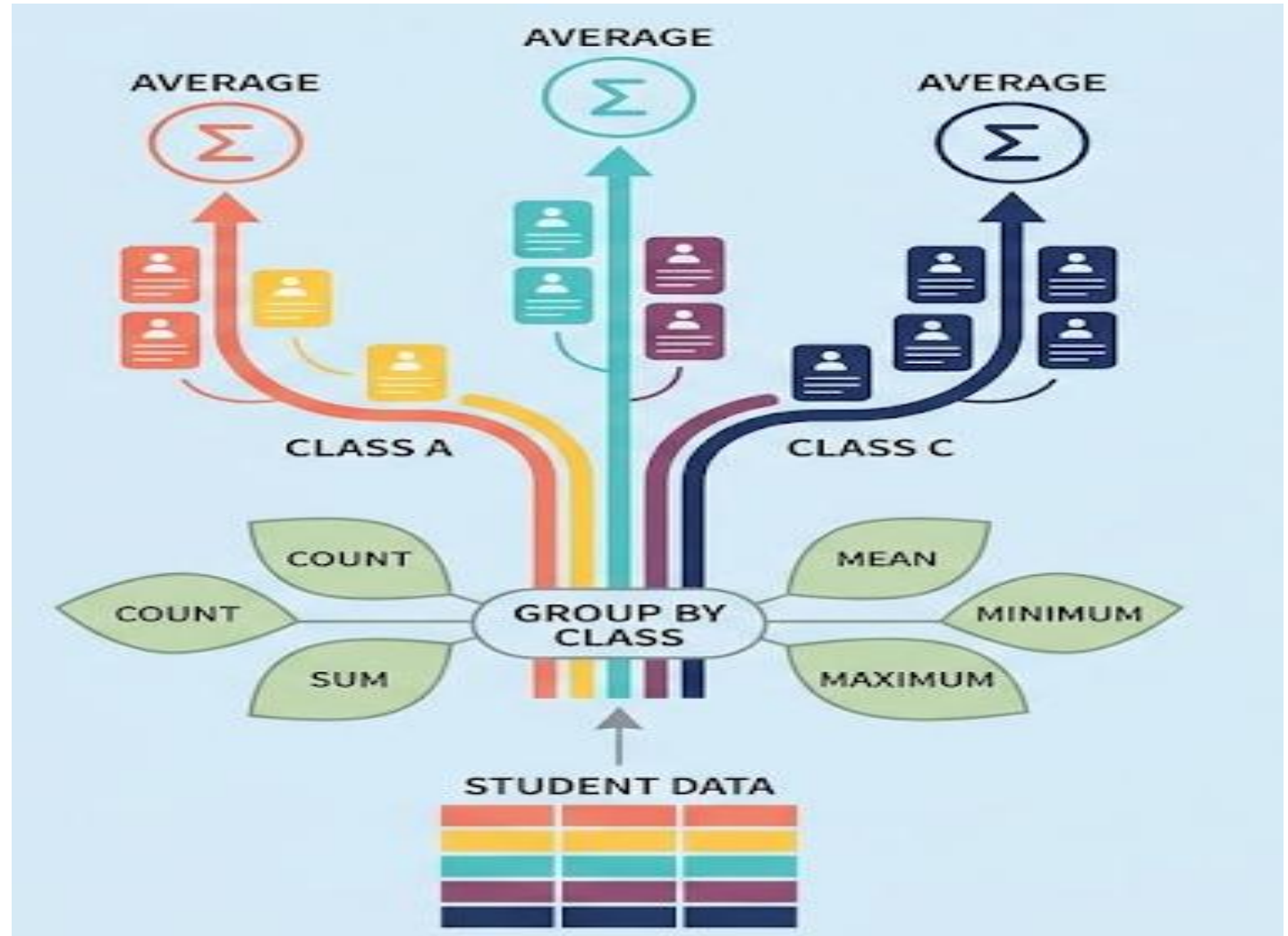
Grouping and Aggregation

Grouping Process

- Choose a category column
- Place matching records into groups
- Apply a calculation to each group
- Compare the grouped results

Common Aggregations

- Count
- Sum
- Mean
- Minimum
- Maximum



Cleaning and Transforming Data

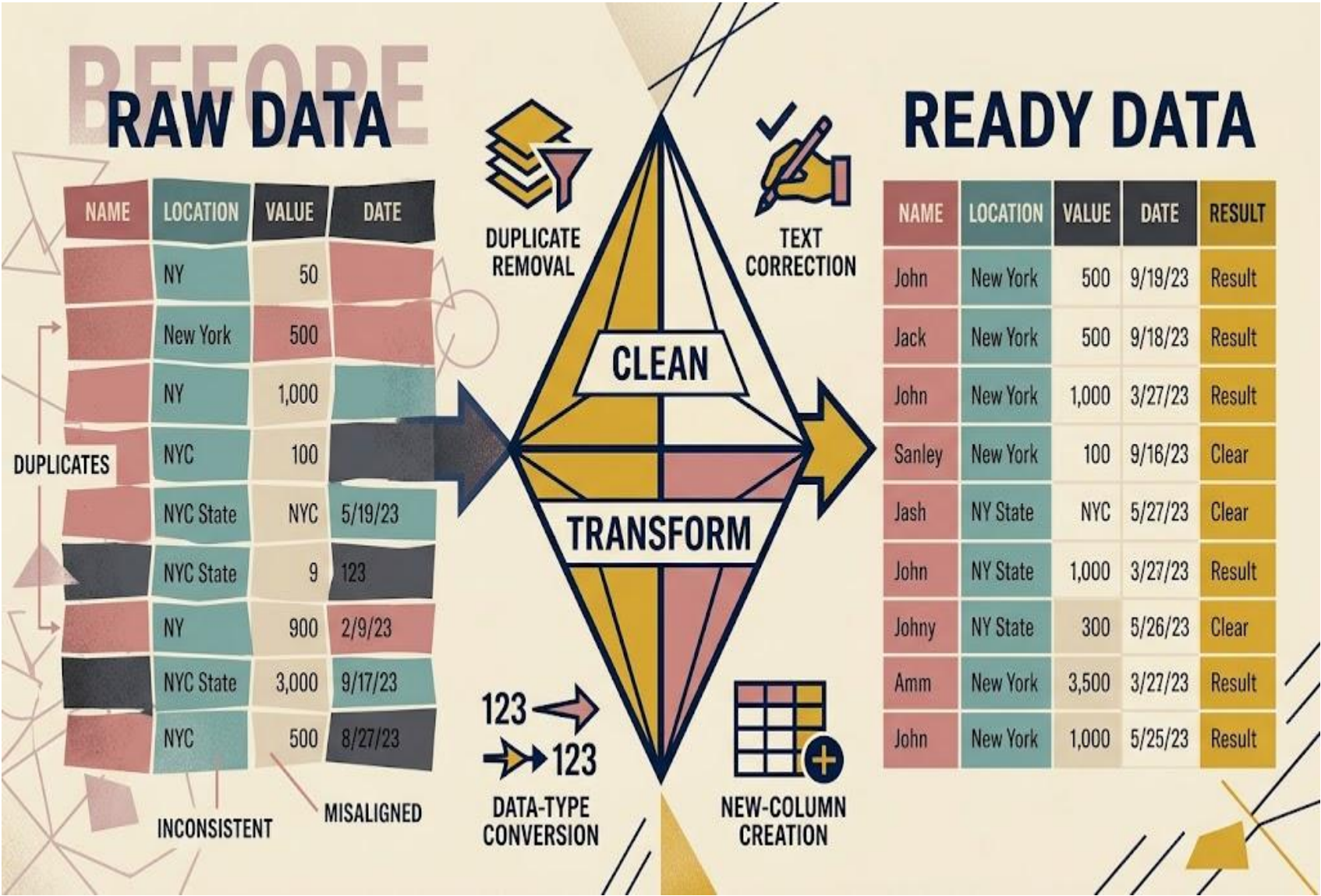
Cleaning fixes problems in a dataset, while transformation changes the data into a more useful form.

Data Cleaning

- Remove duplicate records
- Correct inconsistent values
- Replace invalid entries
- Fix column names

Data Transformation

- Convert data types
- Create new columns
- Update existing values
- Sort and rearrange data



Thank You

Any Question?